

Retrying Made Easy with Tenacity in Python

This article gives a brief overview of the Tenacity library, and how it helps to retry actions in case of transient failures like loss of network.



Image Source: <https://www.freepik.com>

All code should be able to handle transient failures to connect to a service, loss of network, timeouts, etc, gracefully, by providing a retry mechanism that remediates the condition and provides the details of these failures. This remediate mechanism could be as simple as providing a delay before retrying the same action.

Tenacity simplifies the action of retrying behaviour. It offers a decorator based approach, which provides retry behaviour for any condition.

We can easily add the package with a pip installation.

```
pip install tenacity
```

We may need to perform a retry in our code for various reasons. Some might be as simple as looping for a duration or to perform a retry only for a specific error type. Let us discuss the various types of retry approaches provided by the Tenacity library.

Basic retry

A simple retry will help to have a retry whenever an exception is raised. The function retries until there is no exception in the code flow. This approach may not be perfect, but a great one to start experimenting with in the library.

```
@retry
def simple_retry_forever():
    print("simple retry which runs forever")
    raise Exception("Simple retry forever")
```

Retry with time and number of attempts

When developers start working out retries, they typically set a time period or limit the number of retry attempts as part of a loop.

Retry with number of attempts: Here the code tries to retry for a maximum of five attempts before throwing a